# SOLID Principles: Write Code That Doesn't Haunt You at 2 AM

Every developer has been there. You open a file to fix one bug and thirty minutes later you're lost in a web of classes that do too much, methods that know too much, and dependencies so tightly knotted you're afraid to touch anything. The fix for one thing breaks three others.

SOLID principles exist precisely to prevent this. They are five design guidelines — coined by Robert C. Martin (Uncle Bob) — that, when followed, produce code that is easier to understand, extend, and maintain. Not perfect code. Not over-engineered code. Just code that ages well.

Let's break each one down with real examples.

---

## What Is SOLID?

SOLID is an acronym for five object-oriented design principles:

- **S** — Single Responsibility Principle
- **O** — Open/Closed Principle
- **L** — Liskov Substitution Principle
- **I** — Interface Segregation Principle
- **D** — Dependency Inversion Principle

These aren't rules enforced by a compiler. They're judgment calls. The goal is not to follow them religiously — it's to understand *why* they exist so you know when to apply them.

---

## S — Single Responsibility Principle

> *A class should have one, and only one, reason to change.*

This is the simplest principle and the one most frequently violated. When a class does too many things, every change becomes risky because you might break something unrelated.

**Bad example:**

```python
```

```python
class UserManager:
    def create_user(self, data):
        # saves user to database
        pass

    def send_welcome_email(self, user):
        # sends email via SMTP
        pass

    def generate_report(self):
        # generates a PDF report of all users
        pass
```

This class has three reasons to change: database logic changes, email logic changes, or report format changes. They're completely unrelated concerns bundled into one place.

**Better approach:**

```python
class UserRepository:
    def create(self, data):
        # handles database logic only
        pass

class EmailService:
    def send_welcome(self, user):
        # handles email sending only
        pass

class UserReportGenerator:
    def generate(self):
        # handles report logic only
        pass
```

Now each class has one job. If your email provider changes, you touch `EmailService` and nothing else.

**The key question to ask yourself:** *"How many different reasons could force me to edit this class?"* If the answer is more than one, consider splitting it.

---

## O — Open/Closed Principle

> *Software entities should be open for extension, but closed for modification.*

Once a class is working and tested, you should be able to add new behavior without editing its existing code. You extend it — you don't rewrite it.

**Bad example:**

```python
class DiscountCalculator:
    def calculate(self, order, customer_type):
        if customer_type == "regular":
            return order.total * 0.05
        elif customer_type == "premium":
            return order.total * 0.10
        elif customer_type == "vip":
            return order.total * 0.20
```

Every time a new customer type is added, you have to open this class and modify it. This is fragile — you risk introducing bugs in already-working logic.

**Better approach:**

```python
```

```python
from abc import ABC, abstractmethod

class DiscountStrategy(ABC):
    @abstractmethod
    def calculate(self, order):
        pass

class RegularDiscount(DiscountStrategy):
    def calculate(self, order):
        return order.total * 0.05

class PremiumDiscount(DiscountStrategy):
    def calculate(self, order):
        return order.total * 0.10

class VIPDiscount(DiscountStrategy):
    def calculate(self, order):
        return order.total * 0.20

class DiscountCalculator:
    def __init__(self, strategy: DiscountStrategy):
        self.strategy = strategy

    def calculate(self, order):
        return self.strategy.calculate(order)
```

Adding a new discount type means adding a new class — you never touch `DiscountCalculator` again.

**The key question to ask yourself:** *"If requirements change, will I need to edit existing working code or just add new code?"*

---

## L — Liskov Substitution Principle

> *Objects of a subclass should be substitutable for objects of the superclass without breaking the program.*

Named after Barbara Liskov, this principle says that inheritance should mean *"is a"* in a meaningful behavioral sense — not just structurally. If you swap a subclass in where a parent class is expected, the code should still work correctly.

**Bad example:**

```python
```

```python
class Bird:
    def fly(self):
        print("Flying...")

class Penguin(Bird):
    def fly(self):
        raise Exception("Penguins can't fly!")
```

`Penguin` inherits from `Bird`, but calling `.fly()` on it breaks the program. Any code that works with `Bird` objects now has to guard against this case — the substitution doesn't work.

**Better approach:**

```python
class Bird:
    def move(self):
        pass

class FlyingBird(Bird):
    def move(self):
        print("Flying...")

class SwimmingBird(Bird):
    def move(self):
        print("Swimming...")

class Eagle(FlyingBird):
    pass

class Penguin(SwimmingBird):
    pass
```

Now you can pass any `Bird` to code that calls `.move()` and it behaves correctly — no surprises.

**The key question to ask yourself:** *"If I replace the parent class with this subclass everywhere, does the behavior remain sensible?"* If not, the inheritance hierarchy needs rethinking.

---

## I — Interface Segregation Principle

> *Clients should not be forced to depend on interfaces they do not use.*

Fat interfaces — those with many methods — force implementing classes to define methods they don't need. This creates dead code, confusing stubs, and unnecessary coupling.

**Bad example:**

python

```python
from abc import ABC, abstractmethod

class Worker(ABC):
    @abstractmethod
    def work(self):
        pass

    @abstractmethod
    def eat(self):
        pass

    @abstractmethod
    def sleep(self):
        pass

class Robot(Worker):
    def work(self):
        print("Working...")

    def eat(self):
        pass  # Robots don't eat — forced to implement this anyway

    def sleep(self):
        pass  # Robots don't sleep — forced to implement this too
```

**Better approach:**

python

```python
class Workable(ABC):
    @abstractmethod
    def work(self):
        pass

class Eatable(ABC):
    @abstractmethod
    def eat(self):
        pass

class Sleepable(ABC):
    @abstractmethod
    def sleep(self):
        pass

class Human(Workable, Eatable, Sleepable):
    def work(self): print("Working...")
    def eat(self): print("Eating...")
    def sleep(self): print("Sleeping...")

class Robot(Workable):
    def work(self): print("Working...")
```

Each class only depends on what it actually uses. `Robot` never has to pretend it eats.

**The key question to ask yourself:** *"Are any of the classes implementing this interface leaving methods empty or throwing 'not implemented' errors?"* That's a signal the interface needs to be split.

---

## D — Dependency Inversion Principle

> *High-level modules should not depend on low-level modules. Both should depend on abstractions.*

This is perhaps the most architecturally impactful principle. When your business logic (high-level) directly instantiates infrastructure details (low-level) like databases or email clients, swapping those details becomes painful.

**Bad example:**

```python
```

```python
class MySQLDatabase:
    def save(self, data):
        print(f"Saving to MySQL: {data}")

class UserService:
    def __init__(self):
        self.db = MySQLDatabase()  # hardcoded dependency

    def create_user(self, data):
        self.db.save(data)
```

`UserService` is locked to MySQL. Switching to PostgreSQL, or mocking the database in tests, requires editing `UserService` itself.

**Better approach:**

```python
```

```python
from abc import ABC, abstractmethod

class Database(ABC):
    @abstractmethod
    def save(self, data):
        pass

class MySQLDatabase(Database):
    def save(self, data):
        print(f"Saving to MySQL: {data}")

class PostgreSQLDatabase(Database):
    def save(self, data):
        print(f"Saving to PostgreSQL: {data}")

class MockDatabase(Database):
    def save(self, data):
        print(f"Mock save: {data}")

class UserService:
    def __init__(self, db: Database):  # depends on abstraction
        self.db = db

    def create_user(self, data):
        self.db.save(data)

# Usage
service = UserService(MySQLDatabase())
test_service = UserService(MockDatabase())
```

`UserService` now depends on an abstraction — not a concrete implementation. You can swap databases freely, and unit tests become trivial.

**The key question to ask yourself:** *"If I needed to swap out this dependency for a different implementation, how many files would I have to change?"* The answer should be one.

---

## Putting It All Together

SOLID principles aren't a checklist to tick off before shipping. They're lenses — ways of looking at your code and asking "does this design make future changes easier or harder?"

Here's a quick reference:

| Principle | One-line summary |
| --- | --- |
| **Single Responsibility** | One class, one job |
| **Open/Closed** | Add behavior by adding code, not editing it |
| **Liskov Substitution** | Subclasses must be true substitutes for their parents |
| **Interface Segregation** | Prefer small, focused interfaces over fat ones |
| **Dependency Inversion** | Depend on abstractions, not concrete implementations |

The real payoff comes over time. Code written with these principles in mind tends to be easier to test, easier to refactor, and far less likely to cause the kind of cascading failures that turn a two-minute fix into a two-day debugging session.

Start with Single Responsibility — it's the most immediately impactful. Once you feel the difference between a class with one job and a class with five, the others start to click naturally.

---

*Thanks for reading. If this was helpful, feel free to clap and follow for more posts on clean code, design patterns, and software architecture.*